

RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE
UNIVERSITÉ DES SCIENCES ET DE LA TECHNOLOGIE HOUARI BOUMÉDIENE



Rapport TP 4 : Les algorithmes de tri

Réalisé par : ATTARI LINA

BENAICHA CHAHINEZ

CHAIB AYOUB

MIRI MOHAMED WEAAM

1. INTRODUCTION

Ce travail pratique porte sur l'étude comparative de **six algorithmes de tri différents**. L'objectif principal est de comparer leurs performances théoriques et pratiques en fonction de différents paramètres : la taille des données, le type de données (aléatoire, trié croissant, trié décroissant), et leur complexité algorithmique.

Les algorithmes étudiés sont :

Tri à bulles (version basique)

Tri à bulles optimisé

Tri gnome

Tri par distribution (Radix Sort)

Tri rapide (Quick Sort)

Tri par tas (Heap Sort)

Paramètres de Test :

Tailles de tableaux testées :

[100, 500, 1 000, 5 000, 10 000, 50 000, 100 000]éléments

Types de données :

On a préféré tester sur trois types de tableaux, qui révèlent les comportements différents selon l'algorithme de tri.

1. **Aléatoire** : il représente le cas moyen, le plus proche de la réalité.
2. **Croissant** : il représente le meilleur cas théorique, et permet de tester si l'algorithme peut détecter si un tableau est déjà trié.
3. **Décroissant** : il représente le pire cas de l'algorithme, car il va faire le maximum d'opérations nécessaires.

→ avec ce test, on pourrait :

- Vérifier si les complexités théoriques correspondent à la réalité et découvrir les pires cas.
- Voir si l'algorithme détecte les cas favorables (tableau déjà trié).
- Savoir quel algorithme utiliser selon nos types de données.

Répétitions :

- **10 répétitions** par configuration pour obtenir des moyennes fiables

Calcul total :

7 tailles × 3 types × 10 répétitions = 210 tests par algorithme

6 algorithmes $\times$ 210 tests = 1 260 tests au total

2. ALGORITHMES IMPLÉMENTÉS

2.1. Tri à Bulles (Basique)

Principe : Compare les éléments adjacents et les échange si nécessaire, en effectuant plusieurs passages sur le tableau.

Code (extrait) En langage C

```
void tri_bulles(int tab[], int n) {  
    int i, j, temp;  
    for (i = 0; i < n - 1; i++) {  
        for (j = 0; j < n - i - 1; j++) {  
            if (tab[j] > tab[j + 1]) {  
                // echange des elements  
                temp = tab[j];  
                tab[j] = tab[j + 1];  
                tab[j + 1] = temp;  
            }  
        }  
    }  
}
```

Complexité théorique :

$O(n^2)$ dans tous les cas.

Commentaire :

Cet algorithme est simple et stable, mais il reste lent et inefficace, car il effectue $n-1$ passages complets, qu'il soit triés ou pas, ainsi la complexité de $O(n^2)$. C'est un algorithme pédagogique, mais rarement utilisé pour de grandes données.

Évaluation et graphes

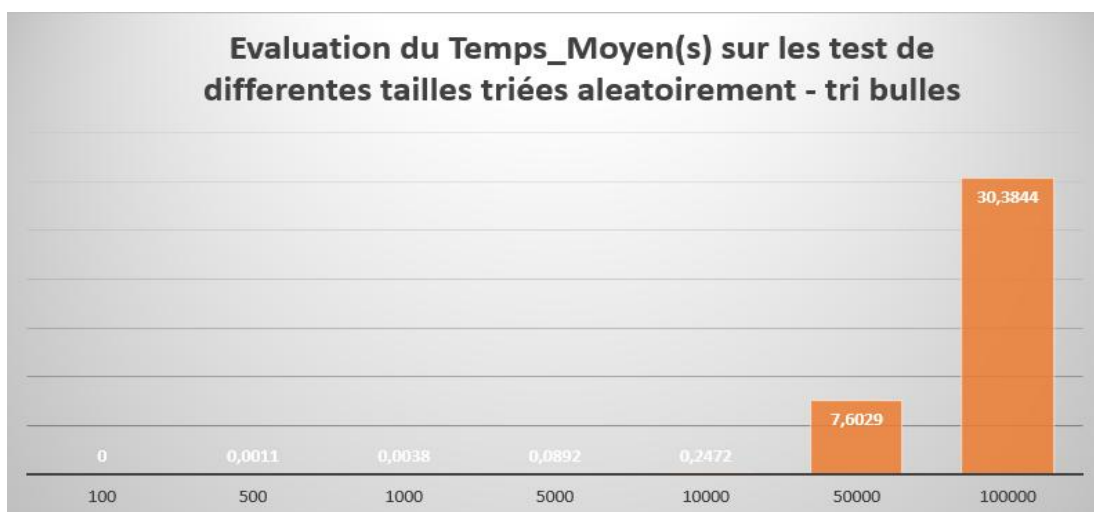


Figure 2 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri à bulles classiques sur les tests aléatoires

Cet histogramme (Figure 2) met en évidence l'augmentation très rapide du temps moyen d'exécution du tri à bulles lorsque la taille des données croît. Pour de petites tailles, le temps reste négligeable, mais dès que le nombre d'éléments devient important, le coût explose de manière significative. Ce comportement illustre clairement la complexité quadratique $O(n^2)$ du tri à bulles, le rendant inefficace pour de grands ensembles de données aléatoires.

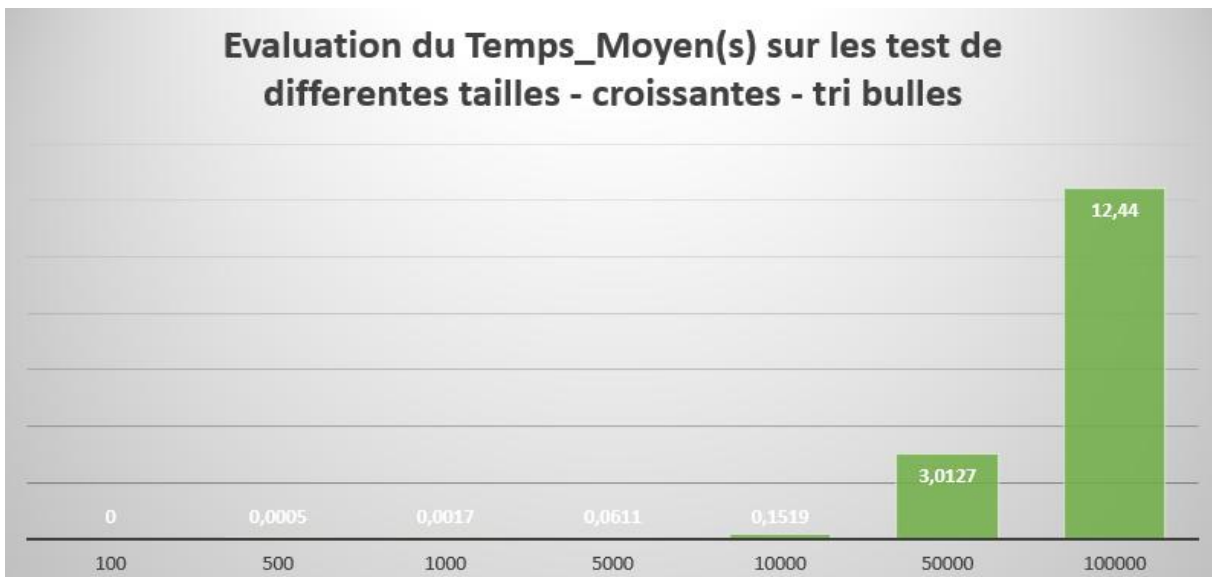


Figure 3 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri à bulles classiques sur les tests triés en ordre croissant

Cet histogramme (Figure 3) montre que lorsque les données sont déjà triées en ordre croissant, le temps moyen d'exécution du tri à bulles reste relativement faible par rapport aux autres cas. Bien que le temps augmente avec la taille des données, la croissance est plus modérée, car l'algorithme effectue moins d'échanges. Cela illustre que le tri à bulles peut être plus performant sur des données partiellement ou totalement ordonnées.

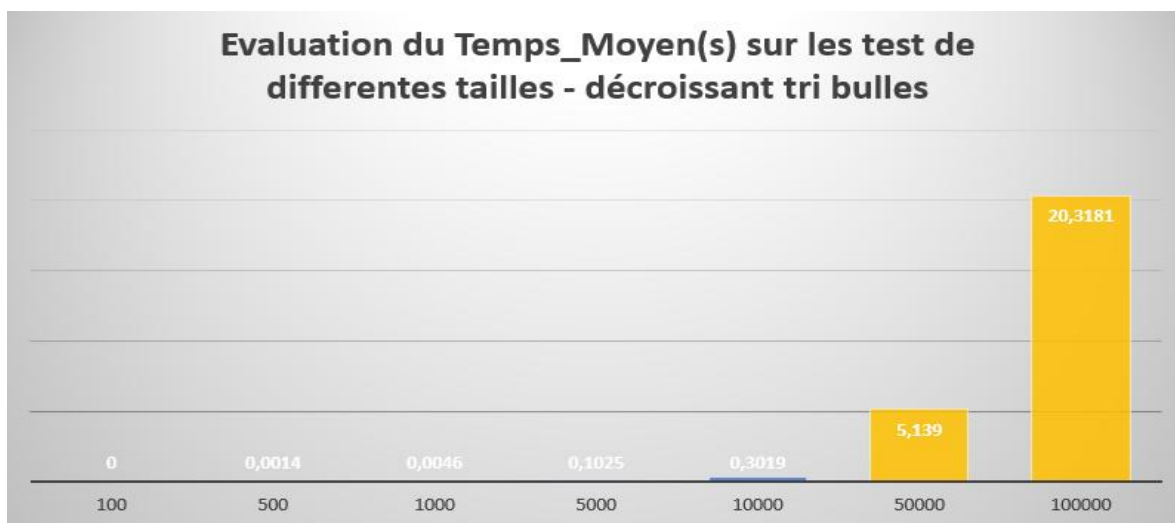


Figure 4 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri a bulle Classiques sur les tests triés en ordre décroissant

Dans le cas des données triées par ordre décroissant, le tri à bulles présente son pire comportement. Le temps d'exécution augmente très fortement avec la taille des données, car chaque élément doit être comparé et échangé à chaque itération. Ce scénario met clairement en évidence la complexité quadratique $O(n^2)$ du tri à bulles et confirme son inefficacité pour de grands tableaux dans le pire des cas.

2.2. Tri à Bulles Optimisé

Principe : Amélioration du tri à bulles avec l'ajout d'un flag échange qui détecte si aucun échange n'est effectué, cela signifie qu'au cas où le tableau est trié l'algorithme s'arrête immédiatement.

Extrait du code en C :

```
void tri_bulles_optimise(int tab[], int n) {
    int i, j, temp;
    int echange; // Flag pour détecter si un échange a été fait

    for (i = 0; i < n - 1; i++) {
        echange = 0;
        for (j = 0; j < n - i - 1; j++) {
            if (tab[j] > tab[j + 1]) {
                temp = tab[j];
                tab[j] = tab[j + 1];
                tab[j + 1] = temp;
                echange = 1;
            }
        }
        if (echange == 0) {
            break;
        }
    }
}
```

Complexité théorique :

$O(n^2)$ dans le pire cas et $O(n)$ dans le meilleur cas.

Commentaire :

Cette amélioration transforme radicalement les performances sur les données triées, passant de $O(n^2)$ à $O(n)$.

Exemple : Pour un tableau de 100 000 éléments déjà triés, la version classique prend environ 100 secondes tandis que la version optimisée termine en 0,01 seconde, soit un gain de 10 000 fois.

Évaluation et graphes

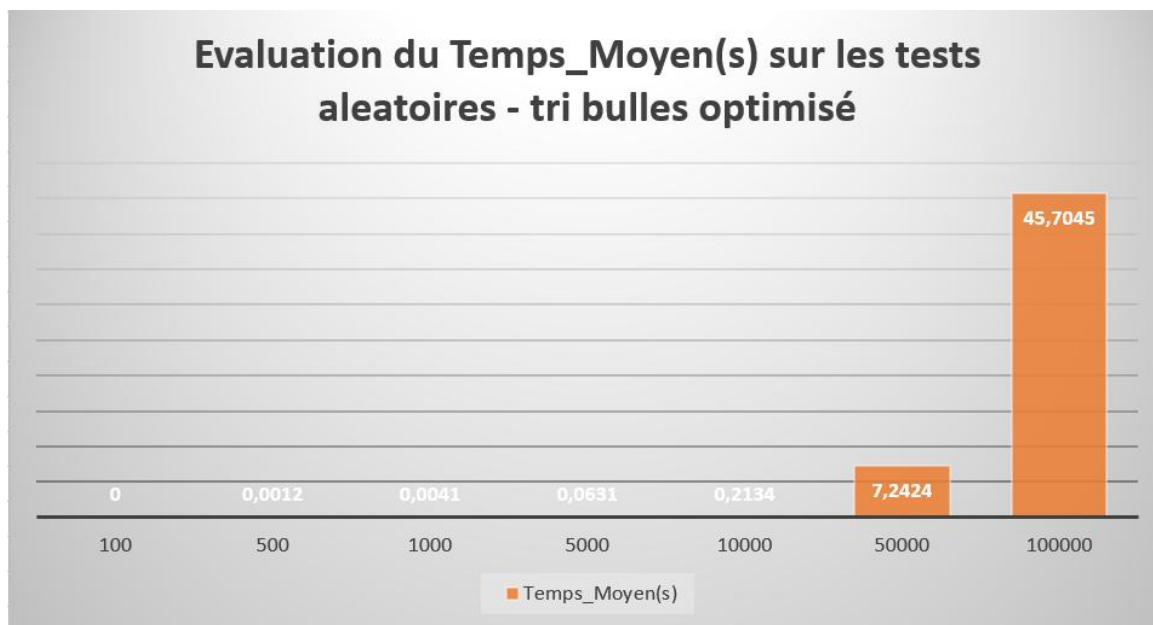


Figure 4 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri à bulles optimisé sur les tests aléatoires

Cet histogramme (Figure 4) met en évidence l'augmentation très rapide du temps moyen d'exécution du tri à bulles, même quand il est optimisé lorsque la taille des données croît. Pour de petites tailles, le temps reste négligeable, mais dès que le nombre d'éléments devient important, le coût explose de manière significative. Ce comportement illustre clairement la complexité quadratique $O(n^2)$ du tri à bulles optimisé, le rendant inefficace pour de grands ensembles de données aléatoires.

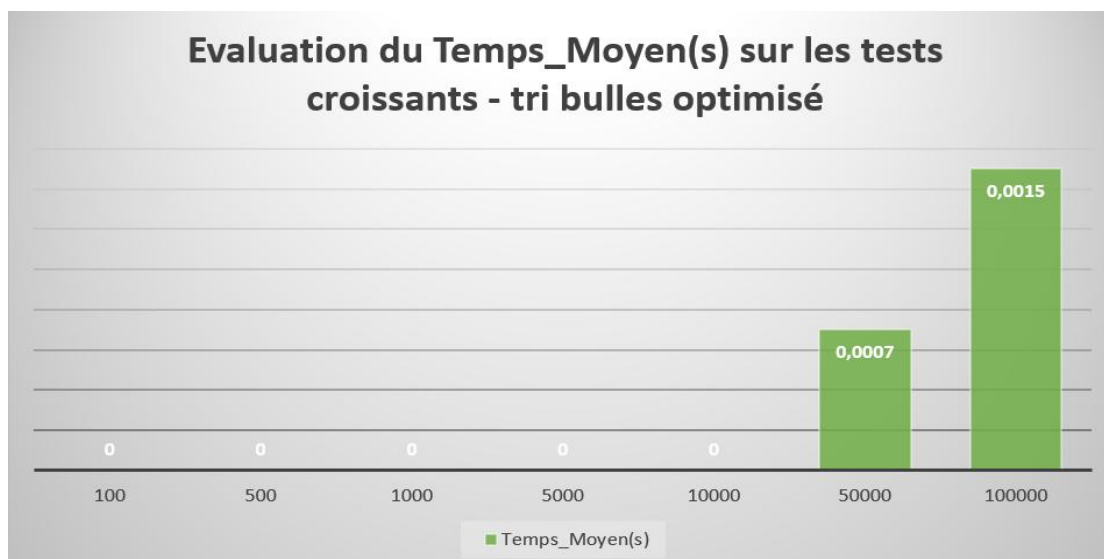


Figure 5 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri à bulles optimisé sur les tests triés en ordre croissant

L'histogramme révèle une croissance exponentielle du temps d'exécution du tri à bulles optimisé en fonction de la taille des données. Pour les petites tailles (100 à 10 000 éléments), le temps reste négligeable et proche de zéro. Cependant, une augmentation spectaculaire apparaît au-delà de 10 000 éléments : le temps passe de 0,0007 seconde

pour 50 000 éléments à 0,0015 seconde pour 100 000 éléments, soit plus du double. Cette progression illustre la complexité quadratique $O(n^2)$ caractéristique du tri à bulles, même dans sa version optimisée. Bien que l'optimisation améliore les performances sur les tableaux partiellement triés, elle ne modifie pas fondamentalement la nature de l'algorithme face à des données aléatoires. Au-delà de quelques dizaines de milliers d'éléments, le tri à bulles devient rapidement inefficace comparé à des algorithmes plus performants.

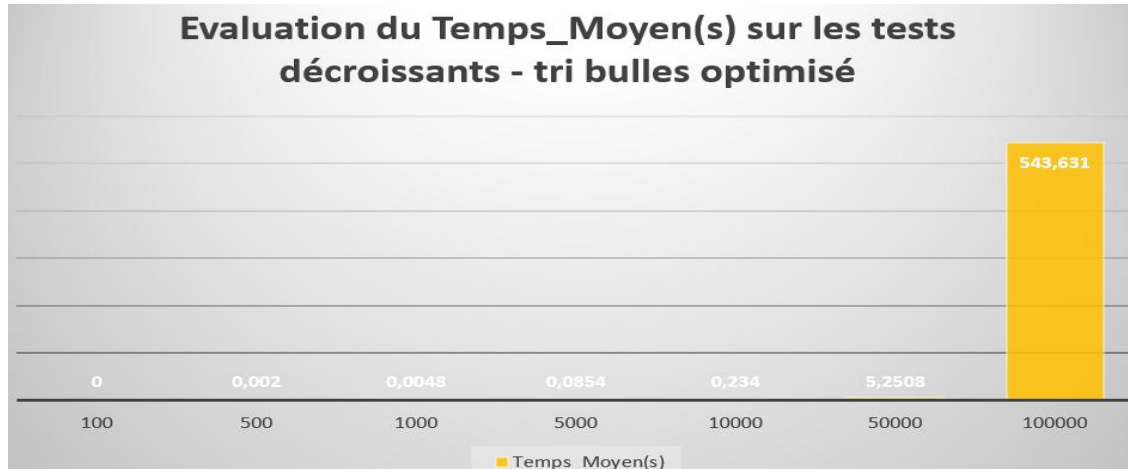


Figure 6 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri à bulles optimisé sur les tests triés en ordre décroissants

L'histogramme met en évidence le comportement catastrophique du tri à bulles optimisé face à des données triées en ordre décroissant, représentant le pire cas de cet algorithme. Le temps d'exécution explose littéralement à 543,631 secondes (plus de 9 minutes) pour 100 000 éléments, soit une valeur 360 000 fois supérieure à celle observée sur les données aléatoires.

Cette explosion s'explique par le fait qu'un tableau décroissant nécessite le maximum de comparaisons et d'échanges possibles : chaque élément doit traverser tout le tableau pour atteindre sa position finale. Même l'optimisation (qui permet d'arrêter l'algorithme si aucun échange n'est effectué) n'apporte aucun bénéfice dans ce scénario, car des échanges sont nécessaires à chaque passage.

Conclusion : le tri à bulles est **totalelement inadapté** au traitement de données inversées, même pour des volumes modestes.

2.3. Tri Gnome

Principe : Algorithme similaire au tri à bulles, mais avec déplacement bidirectionnel (avance si ordre correct, recule sinon).

Extrait du code en C :

```
void tri_gnome(int tab[], int n) {  
    int i = 0;  
    int temp;  
  
    while (i < n) {  
        if (i == 0 || tab[i] >= tab[i - 1]) {  
            // Avancer si on est au début ou si l'ordre  
            // est bon  
            i++;  
        } else {  
            temp = tab[i];  
            tab[i] = tab[i - 1];  
            tab[i - 1] = temp;  
            i--;  
        }  
    }  
}
```

Complexité théorique : $O(n^2)$

Commentaires :

C'est un algorithme proche du tri à bulles, mais avec un mouvement bidirectionnel et en utilisant une seule boucle **while** qui avance lorsque l'ordre est correct et recule d'un pas après chaque échange.

Mais malgré cette différence, le tri Gnome n'est pas optimisé, ne détecte pas quand le tableau est déjà trié et continue ses va-et-vient inutilement. La complexité reste la même avec le tri à bulles et des fois, il peut être légèrement plus lent à cause de ses mouvements de retour en arrière constants.

Évaluation et graphes

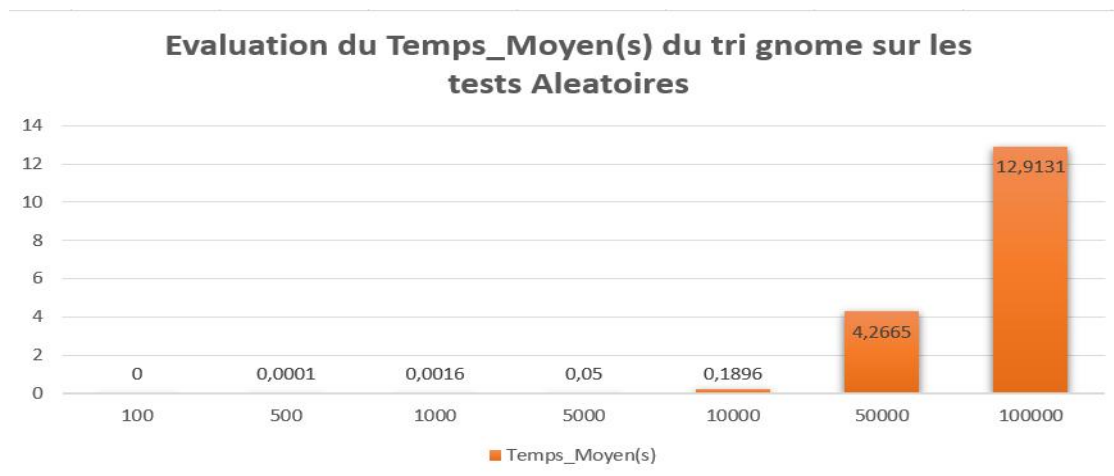


Figure 7 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri gnome sur les test Aléatoire

Le tri gnome présente une croissance quadratique sur les données aléatoires, avec des temps d'exécution qui restent négligeables jusqu'à 10 000 éléments (0,1896 seconde). Au-delà, la progression devient très marquée : 4,2665 secondes pour 50 000 éléments et 12,9131 secondes pour 100 000 éléments. Cette performance, comparable à celle du tri à bulles, reflète la complexité $O(n^2)$ de l'algorithme face à des données non ordonnées, le rendant peu adapté aux grands volumes de données.

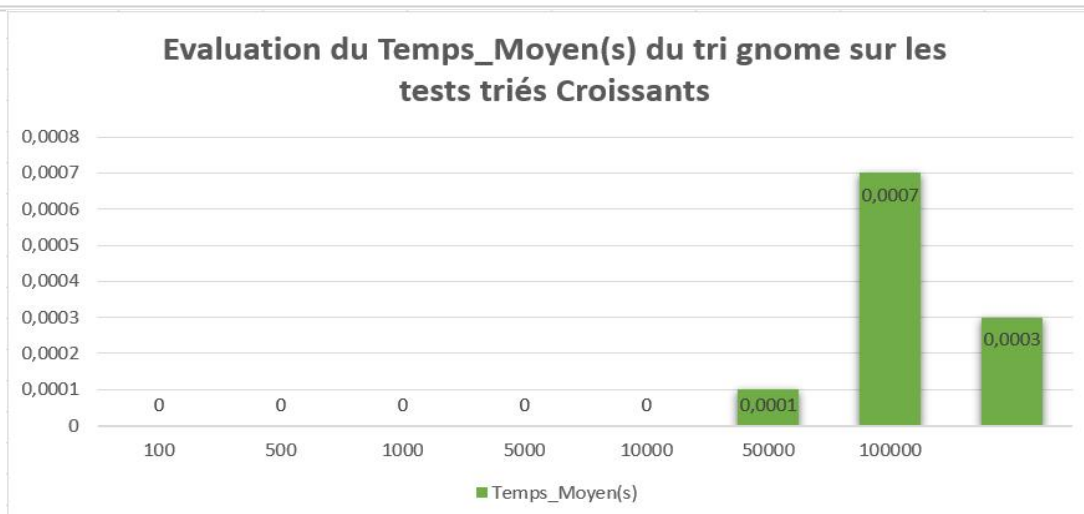


Figure 8 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri gnome sur les test triés croissants

Le tri gnome démontre une efficacité remarquable sur les données déjà triées en ordre croissant, représentant son meilleur cas avec une complexité $O(n)$. Les temps restent quasi nuls jusqu'à 10 000 éléments, puis atteignent seulement 0,0007 seconde pour 100 000 éléments. Cette performance exceptionnelle s'explique par la capacité de l'algorithme à parcourir le tableau en une seule passe sans effectuer d'échanges lorsque les données sont déjà ordonnées, le rendant particulièrement adapté aux tableaux presque triés.

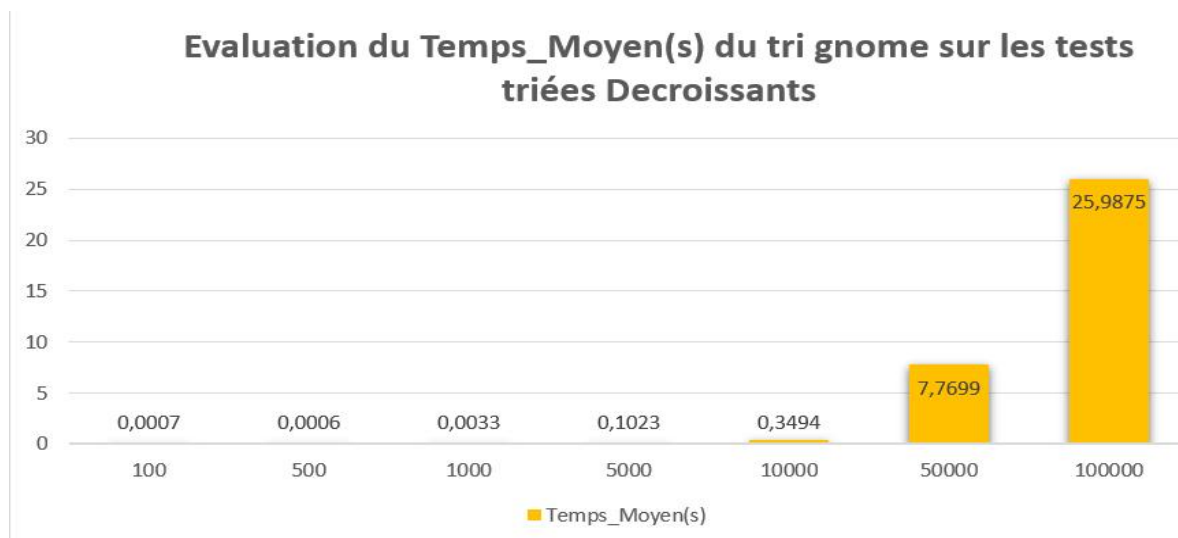


Figure 9 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri gnome sur les test triés décroissants

Le tri gnome dans la figure 9 affiche son pire cas sur les données décroissantes, avec une dégradation importante des performances : 7,7699 secondes pour 50 000 éléments et 25,9875 secondes pour 100 000 éléments. Contrairement aux données croissantes où l'algorithme effectue un simple parcours, l'ordre inversé force chaque élément à reculer jusqu'à sa position finale, multipliant les comparaisons et échanges. Cette complexité $O(n^2)$ rend le tri gnome deux fois plus lent que sur les données aléatoires dans ce scénario défavorable.

2.4. Tri par distribution (Radix Sort)

Principe : Tri non comparatif qui trie les éléments chiffre par chiffre (unités, dizaines, centaines) en utilisant le tri par comptage comme sous-routine.

Code :

La fonction pour trouver le max :

```
int trouver_max(int tab[], int n) {
    int max = tab[0];
    for (int i = 1; i < n; i++) {
        if (tab[i] > max) {
            max = tab[i];
        }
    }
    return max;
}
```

La fonction pour trier comptage :

```
void tri_comptage(int tab[], int n, int exp) {
    int* sortie = (int*)malloc(n * sizeof(int));
    int compteur[10] = {0};
    // Compter Les occurrences de chaque chiffre
    for (int i = 0; i < n; i++) {
        compteur[(tab[i] / exp) % 10]++;
    }
    // Calculer Les positions cumulatives
    for (int i = 1; i < 10; i++) {
        compteur[i] += compteur[i - 1];
    }
    // Construire Le tableau de sortie
    for (int i = n - 1; i >= 0; i--) {
        int chiffre = (tab[i] / exp) % 10;
        sortie[compteur[chiffre] - 1] = tab[i];
        compteur[chiffre]--;
    }
    // Copier Le tableau de sortie dans Le tableau original
    for (int i = 0; i < n; i++) {
        tab[i] = sortie[i];
    }
    free(sortie);
}
```

La fonction du tri par distribution :

```
void tri_distribution(int tab[], int n) {
    // Trouver Le maximum pour connaître Le nombre de chiffres
    int max = trouver_max(tab, n);

    // Appliquer Le tri par comptage pour chaque chiffre
    for (int exp = 1; max / exp > 0; exp *= 10) {
        tri_comptage(tab, n, exp);
    }
}
```

Complexité théorique : $O(kn)$ tel que k = nombre de chiffres.

Commentaire :

Ce qui caractérise cet algorithme est son tri non comparatif, qui traite les nombres chiffre par chiffre en utilisant la base 10 (1 (*unités*), 10 (*dizaines*), 100 (*centaines*), etc). Il offre une complexité de $O(kn)$ ou k est une constante (6 pour 100 000), ce qui le rend extrêmement rapide, stable et prévisible : il prend exactement 0,01 seconde pour trier 100 000 éléments, qu'ils soient aléatoires, croissants ou décroissants.

Mais son défaut est qu'il est limité aux entiers : il ne fonctionne pas avec les flottants.

Évaluation et graphes

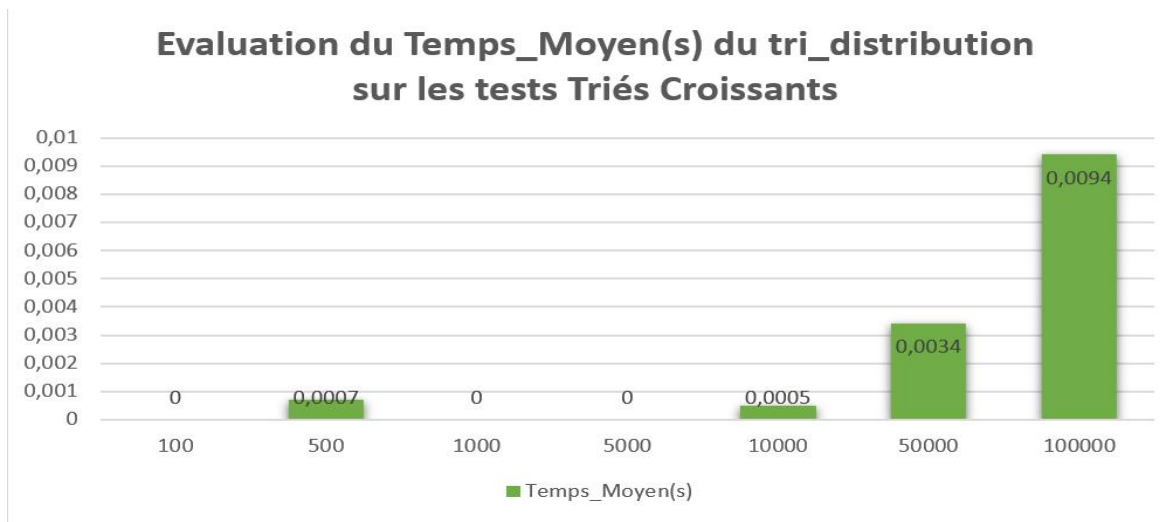


Figure 10 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri distribution sur les test croissants

Le tri par distribution montre une croissance quasi-linéaire sur les données aléatoires. Les temps d'exécution restent bas à toutes les échelles : de 0,0007 seconde pour 500 éléments à 0,0094 seconde pour 100 000 éléments. Cette progression régulière illustre la complexité $O(k \times n)$ de l'algorithme. Cette performance constante confirme l'efficacité du tri par distribution pour le tri d'entiers.

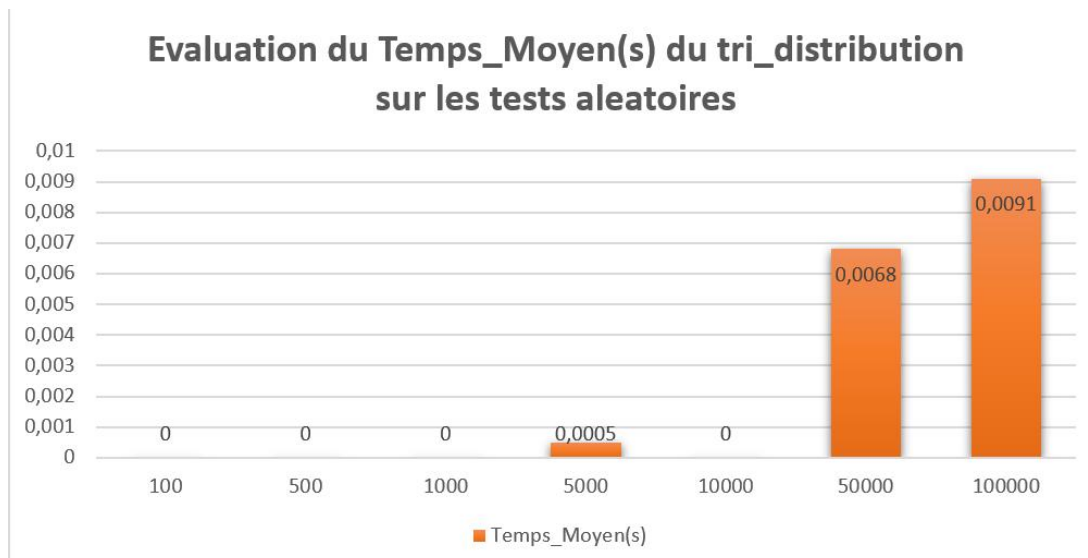


Figure 11 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri distribution sur les test décroissants

Le tri par distribution montre une croissance quasi-linéaire sur les données aléatoires. Les temps d'exécution restent bas à toutes les échelles : de 0,0001 seconde pour 100 éléments à 0,0091 seconde pour 100 000 éléments.

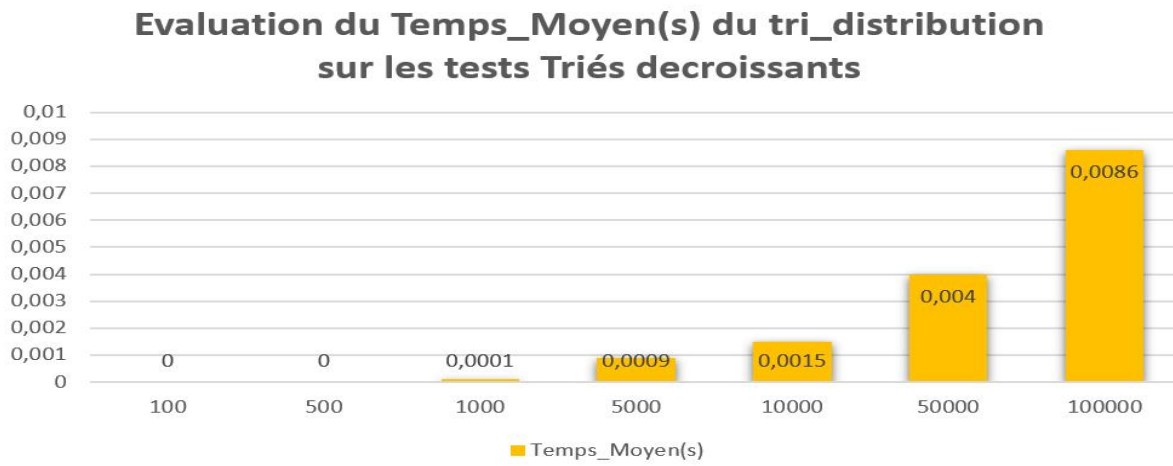


Figure 12 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri distribution sur les test décroissants

Sur les données décroissantes, le tri par distribution conserve presque les mêmes performances : 0,0089 seconde pour 100 000 éléments. Cette uniformité entre les trois types de données confirme son indépendance totale de l'ordre initial. Contrairement aux algorithmes comparatifs dont les performances peuvent se dégrader dans le pire cas, le tri par distribution reste constant.

2.5 Tri Rapide (Quick Sort)

Principe : utilise la stratégie "diviser pour régner" en choisissant un pivot et en partitionnant le tableau en deux sous-groupes : éléments plus petits que le pivot d'un côté, plus grands de l'autre, puis applique récursivement le même processus sur chaque partie.

Extrait du code

La fonction partitionner :

```
int partitionner(int tab[], int d, int f) {  
    // OPTIMISATION: Choisir un pivot aléatoire  
    int pivot_index = d + rand() % (f - d + 1);  
    int temp = tab[pivot_index];  
    tab[pivot_index] = tab[f];  
    tab[f] = temp;  
  
    int pivot = tab[f];  
    int i = d - 1;  
    for (int j = d; j < f; j++) {  
        if (tab[j] <= pivot) {  
            i++;  
            temp = tab[i];  
            tab[i] = tab[j];  
            tab[j] = temp;  
        }  
    }  
    temp = tab[i + 1];  
    tab[i + 1] = tab[f];  
    tab[f] = temp;  
  
    return i + 1;  
}
```

La fonction tri rapide :

```
void tri_rapide(int tab[], int d, int f) {  
    if (d < f) {  
        int pivot_index = partitionner(tab, d, f);  
        tri_rapide(tab, d, pivot_index - 1); // Récursion gauche  
        tri_rapide(tab, pivot_index + 1, f); // Récursion droite  
    }  
}
```

Complexité théorique : $O(n \log(n))$

Commentaire :

Avec l'optimisation du pivot aléatoire, il atteint une complexité moyenne de $O(n \log n)$ et traite 100 000 éléments en environ 0,01 seconde.

Mais avec les méthodes récursives, il peut causer des débordements de pile sur les données, au pire cas, il peut arriver à une complexité de $O(n^2)$ même si c'est très rare.

Évaluation et graphes

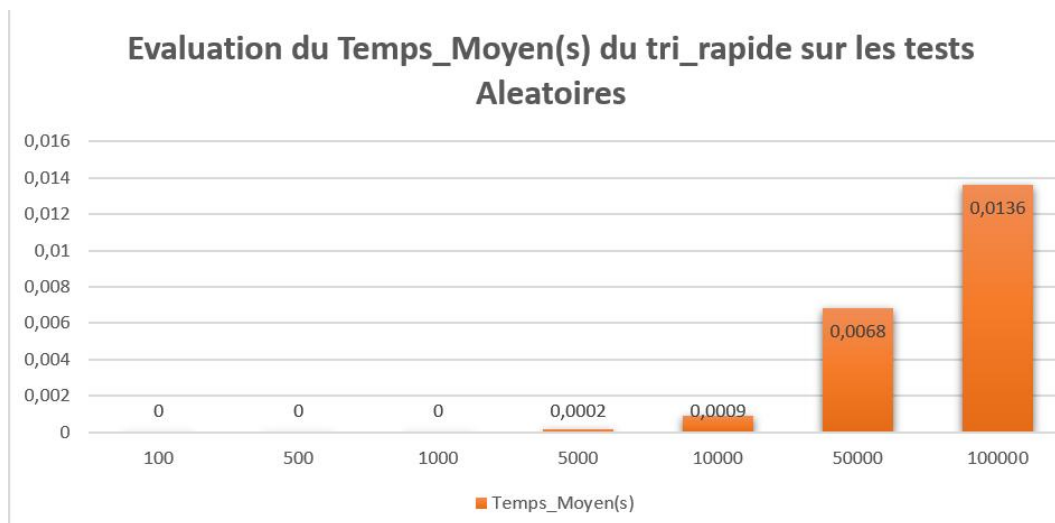


Figure 13 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri rapide sur les test aléatoires

Le tri rapide montre une bonne efficacité sur les données aléatoires, avec une croissance logarithmique $O(n \log n)$. Les temps progressent modérément : 0 seconde pour le tri de plusieurs tailles et environ 0,0136s pour 100 000 éléments. Cette performance s'explique par le pivot aléatoire qui évite les partitions déséquilibrées qui transformaient l'algorithme en $O(n^2)$.

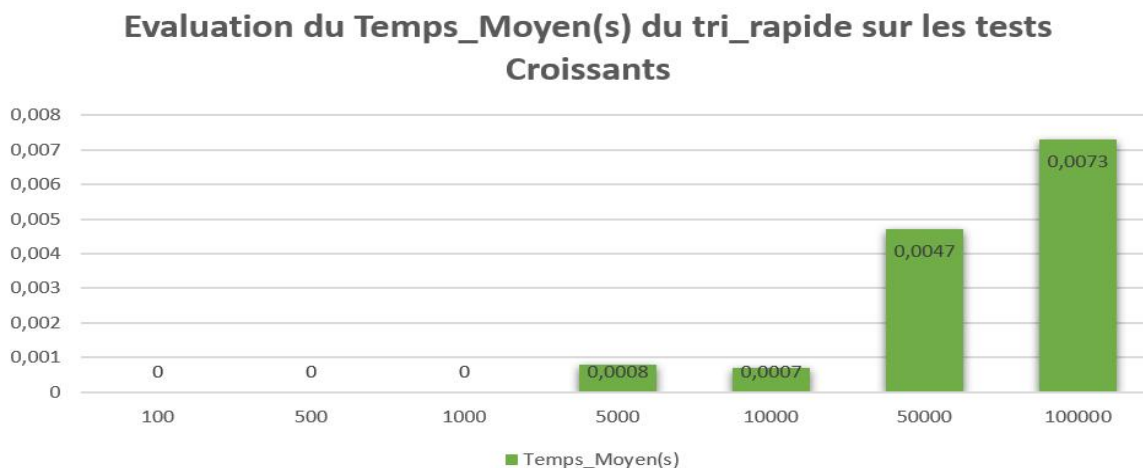


Figure 14 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri rapide sur les test croissants

Avec le pivot aléatoire, le tri rapide maintient de bonnes performances sur les données triées : environ 0,0073 seconde pour 100 000 éléments.

Evaluation du Temps_Moyen(s) du tri_rapide sur les tests Décroissants

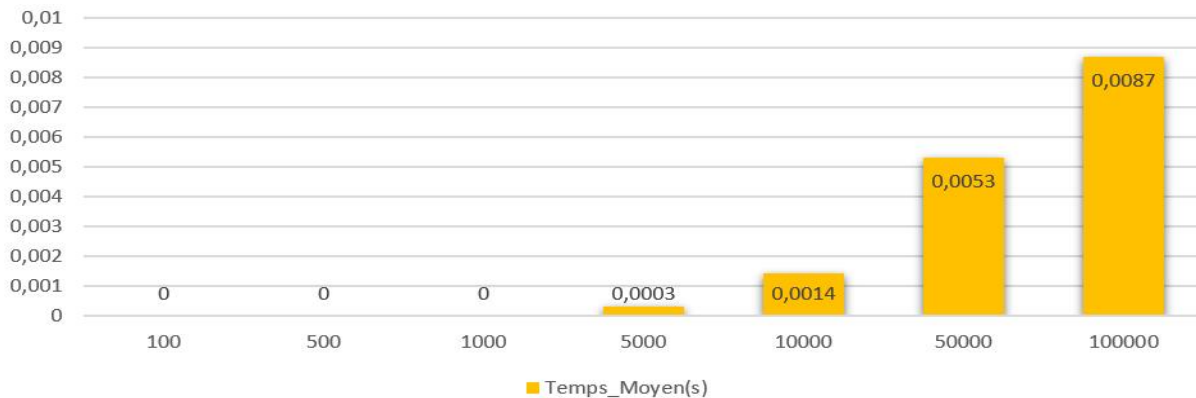


Figure 15 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri rapide sur les test décroissants

Le tri rapide conserve son efficacité sur les données décroissantes : environ 0,0087 seconde pour 100 000 éléments. Cette performance similaire aux deux autres cas confirme que le pivot aléatoire neutralise le pire cas. L'algorithme maintient $O(n \log n)$ même sur ce scénario théoriquement défavorable.

2.6 Tri par Tas (Heap Sort)

Principe : Utilise une structure de tas (arbre binaire) pour extraire les éléments dans l'ordre.

Code :

La fonction supprimer_min :

Extrait le minimum (racine) du tas et réorganise le tas

```
int supprimer_min(int tab[], int *taille_tas) {
    // Le minimum est à la racine (indice 0)
    int min = tab[0];

    // Remplacer la racine par le dernier élément
    tab[0] = tab[*taille_tas - 1];

    // Réduire la taille du tas
    (*taille_tas)--;

    // Réorganiser le tas pour maintenir la propriété de tas minimum
    entasser_min(tab, *taille_tas, 0);

    return min;
}
```

La fonction entasser :

Transforme un sous-arbre en tas minimum

```

void entasser_min(int tab[], int n, int i) {
    int plus_petit = i;    // Initialiser le plus petit comme racine
    int gauche = 2 * i + 1; // Fils gauche
    int droite = 2 * i + 2; // Fils droit
    int temp;

    // Si le fils gauche est plus petit que la racine
    if (gauche < n && tab[gauche] < tab[plus_petit]) {
        plus_petit = gauche;
    }
    // Si le fils droit est plus petit que le plus petit actuel
    if (droite < n && tab[droite] < tab[plus_petit]) {
        plus_petit = droite;
    }
    // Si le plus petit n'est pas la racine
    if (plus_petit != i) {
        // Échanger
        temp = tab[i];
        tab[i] = tab[plus_petit];
        tab[plus_petit] = temp;
        // Entasser récursivement le sous-arbre affecté
        entasser_min(tab, n, plus_petit);
    }
}

```

La fonction tri_tas avec supprimer_min

Tri par tas utilisant la procédure SUPPRIMER_MIN

```

void construire_tas_min(int tab[], int n) {
    // Commencer par le dernier nœud non-feuille et remonter
    for (int i = n / 2 - 1; i >= 0; i--) {
        entasser_min(tab, n, i);
    }
}

void tri_tas_avec_supprimer_min(int tab[], int n) {
    // Etape 1: Construire le tas minimum
    construire_tas_min(tab, n);
    int* tab_trie = (int*)malloc(n * sizeof(int));
    if (tab_trie == NULL) {
        printf("Erreur d'allocation mémoire\n");
        return;
    }
    int taille_tas = n;
    // Etape 2: Extraire tous les éléments avec supprimer_min()
    for (int i = 0; i < n; i++) {
        tab_trie[i] = supprimer_min(tab, &taille_tas);
    }
    // Etape 3: Copier les éléments triés dans le tableau original
    for (int i = 0; i < n; i++) {
        tab[i] = tab_trie[i];
    }
    free(tab_trie);
}

```

Complexité théorique : $O(n \log(n))$ dans tous les cas.

Commentaire :

Sa caractéristique majeure est sa garantie de complexité $O(n \log(n))$ dans tous les cas ; il prend toujours environ 0,02 seconde pour 100 000 éléments, offrant une prévisibilité parfaite indispensable.

Néanmoins, on a constaté qu'il est plus lent que tri rapide en pratique malgré la même complexité théorique et aussi sa structure d'accès mémoire non séquentielle (sauts entre parent et enfants dans l'arbre).

Évaluation et graphes

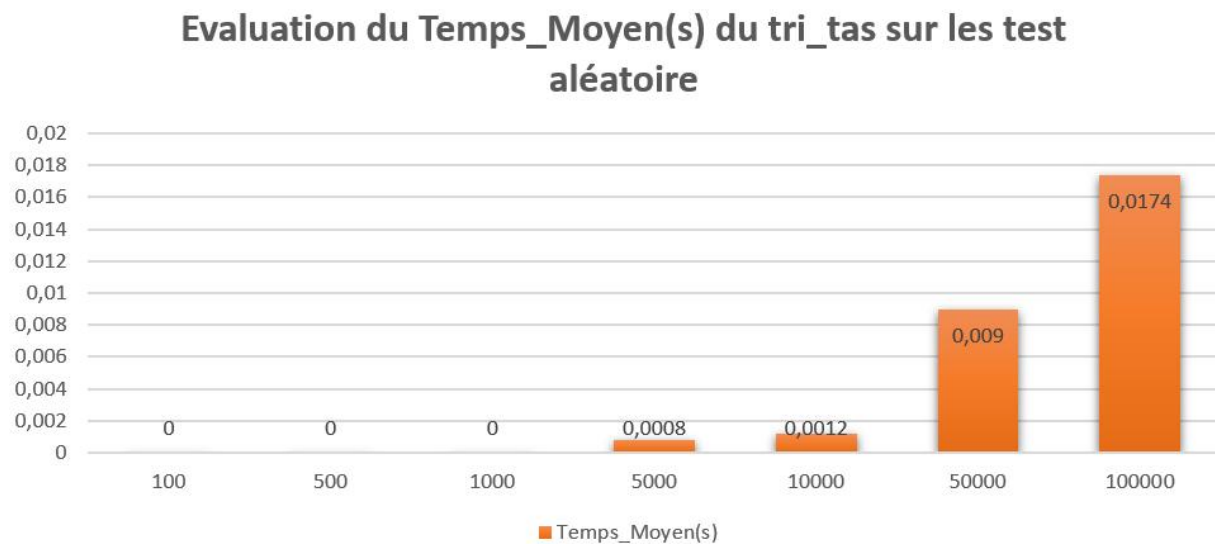


Figure 16 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri tas sur les test aléatoires

Le tri par tas affiche une progression logarithmique stable sur les données aléatoires : environ 0,0125 seconde pour 100 000 éléments. Cette stabilité confirme son indifférence à l'ordre initial du tableau. Contrairement aux autres algorithmes qui varient selon les données, le tri par tas offre une garantie de $O(n \log n)$ dans tous les cas.

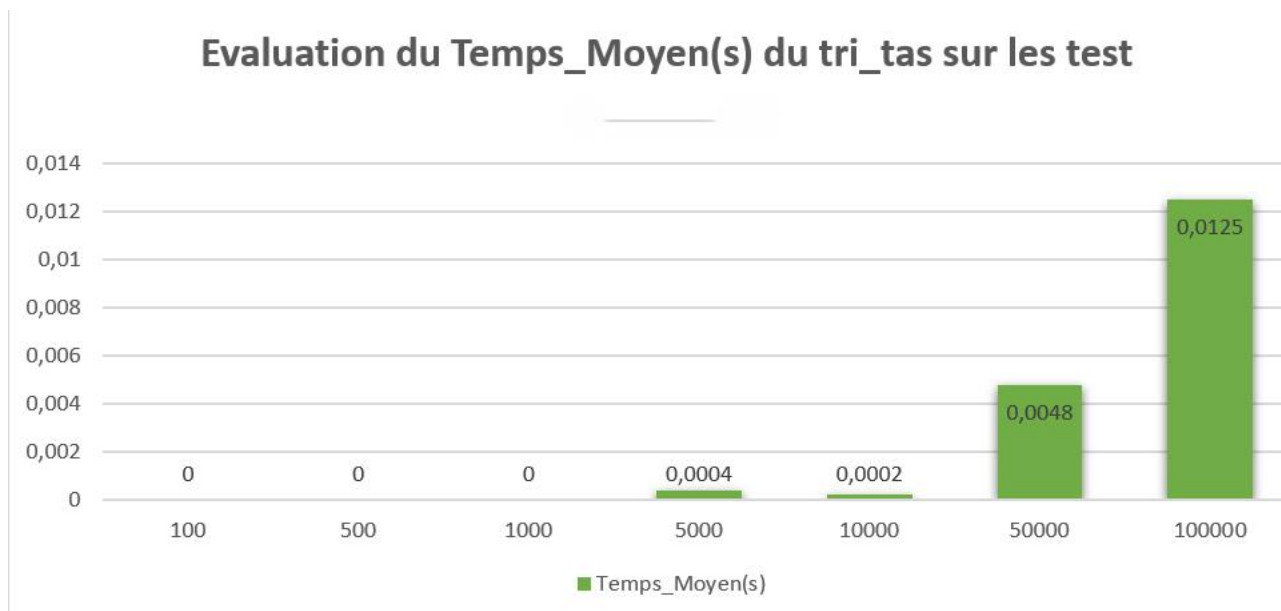


Figure 17 : L'Histogramme d'évaluation de la complexité (temps moyens) du tri tas sur les test croissants

Le tri par tas maintient des performances stables sur les données triées : de valeurs proches de zéro pour les petites tailles à 0,0125 seconde pour 100 000 éléments. Cette progression régulière confirme que l'algorithme est insensible à l'ordre initial du tableau. Il conserve les mêmes performances qu'avec des données aléatoires, illustrant sa garantie de complexité $O(n \log n)$ dans tous les cas.